LAB 06: QUẢN LÝ TIẾN TRÌNH TRONG LINUX

Hệ Điều Hành & Lập Trình Hệ Thống Unix/Linux • Tài Liệu Hướng Dẫn Lý Thuyết

PHẦN I: TỔNG QUAN

I. MỤC TIÊU BÀI HỌC (OBJECTIVES)

- Nắm vững khái niệm về **chương trình (program)**, **tiến trình (process)** và vòng đời thực thi của tiến trình trong nhân Linux.
- Hiểu rõ cơ chế phân chia không gian địa chỉ bộ nhớ ảo (Text, Data, BSS, Heap, Stack) và bảng trang (Page Table / MMU).
- Làm chủ các lời gọi hệ thống cốt lõi: `fork()`, `execve()`, `exit()`, `wait()`, `waitpid()`.
- Nắm vững cơ chế giao tiếp liên tiến trình (IPC) cơ bản: Tín hiệu hệ thống (Signals) và Đường ống dữ liệu (Pipes).

II. LÝ THUYẾT CƠ BẢN (BASIC THEORY)

1. Giới thiệu về Tiến trình (Introduction to Process)

Chương trình (Program) là một tập hợp các chỉ thị được viết bằng ngôn ngữ lập trình nhằm thực thi một tác vụ cụ thể, được lưu trữ thụ động trên đĩa từ (passive entity on disk).
Tiến trình (Process) là một thực thể đang hoạt động (active entity) — chính là một phiên bản thực thi (instance) của chương trình được nạp vào bộ nhớ RAM, sở hữu không gian địa chỉ riêng và được cấp phát tài nguyên hệ thống (CPU, bộ nhớ, I/O).

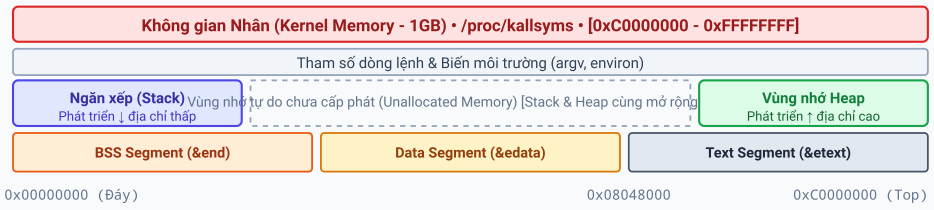
Đặc tính (Feature)	Chương trình (Program)	Tiến trình (Process)
Định nghĩa & Trạng thái	Tập chỉ thị mã máy lưu trên ổ đĩa. Thụ động (Passive, không hoạt động).	Chương trình đang chạy trong bộ nhớ RAM. Chủ động (Active entity).
Vị trí lưu trữ	Tệp thực thi (executable binary) lưu trên ổ đĩa.	Nạp trong bộ nhớ RAM và thanh ghi CPU suốt thời gian thực thi.
Vòng đời (Lifespan)	Lâu dài, tồn tại cố định cho đến khi bị xóa khỏi đĩa.	Tạm thời, bắt đầu khi nạp chạy và kết thúc khi giải phóng.
Sử dụng tài nguyên	Không tiêu tốn chu kỳ CPU, không chiếm dụng RAM hay I/O.	Được cấp phát thời gian CPU, bộ nhớ, bộ mô tả tệp (File Descriptors).
Tính đa bản thể	Chỉ cần một bản nhị phân duy nhất lưu trên đĩa lưu trữ.	Có thể tạo nhiều tiến trình đồng thời từ cùng một tệp chương trình.

Cấu Trúc Không Gian Bộ Nhớ Của Tiến Trình (Memory Layout)

Khi tiến trình khởi tạo, nhân Linux cấp phát không gian địa chỉ ảo và phân định thành các phân vùng (segments) nhằm quản lý việc thực thi mã lệnh và tổ chức dữ liệu một cách tối ưu:

Phân vùng (Segment)	Mô tả chức năng	Hành vi kích thước	Ví dụ mã C
Text (Code)	Chứa mã máy (machine code instructions) sau biên dịch. Chỉ đọc (Read-only) để chống ghi đè lỗi.	Kích thước cố định (Fixed size).	`int main() { ... }`
Data Segment	Lưu trữ các biến toàn cục (global) và biến tĩnh (static) *đã được khởi tạo* giá trị khác 0.	Kích thước cố định.	`int x = 10; static int y = 20;`
BSS Segment	Lưu các biến toàn cục và biến tĩnh *chưa khởi tạo* (hoặc gán = 0). Kernel tự động đặt về 0.	Kích thước cố định.	`int uninit_var; static int count;`
Heap Segment	Vùng nhớ cấp phát động trong thời gian chạy (runtime) thông qua `malloc()`, `calloc()`.	Phát triển hướng lên trên (vùng địa chỉ cao).	`int *p = malloc(sizeof(int));`
Stack Segment	Lưu khung hàm (stack frames), biến cục bộ (local variables), tham số hàm và địa chỉ trả về.	Phát triển hướng xuống dưới (vùng địa chỉ thấp).	`void f() { int local; }`

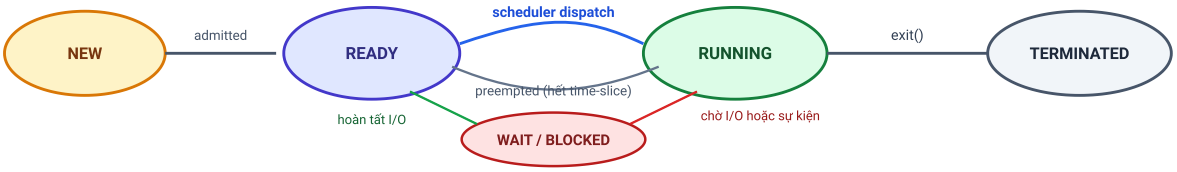
Hình 6-1: Bố cục không gian địa chỉ ảo chuẩn của tiến trình trên Linux (x86-32)



Vòng Đời Và Các Trạng Thái Của Tiến Trình Trong Linux

Trong suốt chu kỳ hoạt động, một tiến trình lần lượt chuyển dịch qua các trạng thái xác định:

- **New (Mới tạo):** Tiến trình vừa được khởi tạo bởi hệ thống nhưng chưa được đưa vào hàng đợi lập lịch.
- **Ready (Sẵn sàng):** Tiến trình đã nạp đủ vào RAM và nằm trong hàng đợi sẵn sàng (Runqueue), chờ CPU cấp phát thời gian chạy.
- **Running (Đang thực thi):** Tiến trình hiện đang được CPU xử lý các chỉ thị mã lệnh.
- **Blocked / Waiting (Chờ / Bị khóa):** Tiến trình tạm dừng chờ một sự kiện (hoàn tất I/O, tín hiệu, khóa tài nguyên).
- **Terminated (Kết thúc):** Tiến trình dừng thực thi, hoàn trả bộ nhớ; chờ tiến trình cha đọc trạng thái kết thúc.



2. CÁC LỜI GỌI HỆ THỐNG CƠ BẢN LÀM VIỆC VỚI TIẾN TRÌNH

Vòng đời của tiến trình được điều khiển thông qua bộ tứ lời gọi hệ thống cốt lõi: `fork()` (tạo tiến trình bản sao), `execve()` (nạp chương trình mới), `exit()` (kết thúc tiến trình) và `wait()` (đồng bộ hóa và thu nhận trạng thái).

KHỞI TẠO TIẾN TRÌNH & CƠ CHẾ HOẠT ĐỘNG CỦA FORK()

Cơ Chế Nhân Bản Tiến Trình • Sao Chép Bộ Nhớ • Xử Lý Phân Nhánh Cha/Con

PHẦN III: FORK() & ĐỒNG BỘ

2.1 Tổng Quan Sự Phối Hợp: fork(), exit(), wait() Và execve()

1. **Tiến trình cha (Parent)** gọi `fork()` để sinh ra một **tiến trình con (Child)** bản sao hoàn chỉnh.
2. Tiến trình con có thể tiếp tục thực thi mã nguồn cũ, hoặc gọi `execve()` để nạp đè một file thực thi mới vào toàn bộ không gian nhớ.
3. Khi thực hiện xong nhiệm vụ, con gọi `exit(status)` để giải phóng tài nguyên và trả mã kết thúc cho kernel.
4. Tiến trình cha gọi `wait(&status)` sẽ bị đình chỉ (suspend) cho tới khi tiến trình con kết thúc, sau đó thu hồi thông tin dọn dẹp bảng tiến trình.

2.2 KHỞI TẠO TIẾN TRÌNH MỚI: LỜI GỌI HỆ THỐNG FORK()

Lời gọi hệ thống `fork()` tạo ra một tiến trình con là bản sao gần như chính xác của tiến trình gọi (tiến trình cha).

```
// Cú pháp hàm fork() từ thư viện chuẩn <unistd.h>
#include <unistd.h>
pid_t fork(void);

/* Ý NGHĨA GIÁ TRỊ TRẢ VỀ:
   • Trong tiến trình CHA : Trả về PID của con vừa tạo (giá trị > 0)
   • Trong tiến trình CON : Nhận giá trị bằng 0
   • Khi xảy ra lỗi       : Trả về -1 (không tạo được con, gán mã lỗi vào errno) */
```

Các đặc điểm mấu chốt của fork():

- **Tính phân nhánh kép (Dual Return):** Hàm `fork()` chỉ được gọi 1 lần nhưng trả về 2 lần ở hai ngữ cảnh khác nhau. Ngay sau khi hoàn tất, hai tiến trình độc lập cùng tồn tại và tiếp tục chạy từ dòng lệnh ngay sau lệnh gọi `fork()`.
- **Nhân bản không gian nhớ:** Tiến trình con nhận bản sao riêng biệt của các phân vùng Stack, Data, Heap. Khi một bên sửa biến, bên kia hoàn toàn không bị ảnh hưởng (nhờ cơ chế *Copy-On-Write* của hệ điều hành).
- **Chia sẻ tài nguyên:** Con kế thừa toàn bộ các bộ mô tả tệp mở (File Descriptors) và vị trí con trỏ tệp (file offset) từ cha.
- **Tính bất định trong lập lịch (Non-deterministic Scheduling):** Không thể biết trước tiến trình cha hay tiến trình con sẽ được CPU phân phối thời gian chạy trước.

```
/* Ví dụ mẫu: Cấu trúc phân nhánh thực thi giữa Cha và Con */
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t childPid = fork();  // Tách dòng thực thi thành 2 tiến trình

    if (childPid < 0) {
        perror("Khởi tạo tiến trình thất bại");
        return 1;
    } else if (childPid == 0) {
        // ================= KHỐI CỦA TIẾN TRÌNH CON =================
        printf("[TIẾN TRÌNH CON] PID = %d | PID Cha = %d\n", getpid(), getppid());
        _exit(0);
    } else {
        // ================= KHỐI CỦA TIẾN TRÌNH CHA =================
        printf("[TIẾN TRÌNH CHA] PID = %d | Đã sinh ra con có PID = %d\n", getpid(), childPid);
    }
    return 0;
}
```

2.3 ĐỒNG BỘ HÓA VÀ CHỜ TIẾN TRÌNH CON KẾT THÚC

Lời Gọi Hệ Thống wait()

LỜI GỌI HỆ THỐNG WAITPID() & KIỂM SOÁT TRẠNG THÁI

Giám Sát Trạng Thái Thoát • Bitfield Status 16-bit • Các Macro Phân Tích sys/wait.h

PHẦN IV: WAITPID() & STATUS

Lời Gọi Hệ Thống Nâng Cao: waitpid()

`waitpid()` là phiên bản mở rộng linh hoạt hơn `wait()`, cho phép cha chọn đích danh tiến trình con cần chờ, kiểm tra trạng thái không chặn (non-blocking), hoặc giám sát khi con bị dừng/tiếp tục.

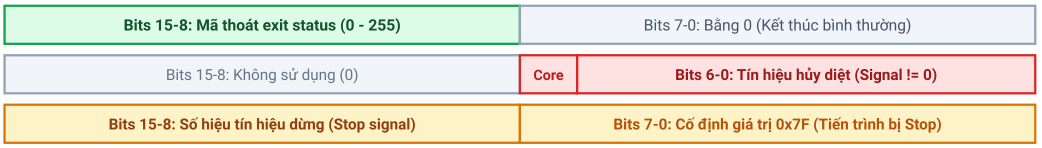
```
#include <sys/wait.h>
pid_t waitpid(pid_t pid, int *status, int options);
/* pid: >0 (chờ đúng PID), -1 (chờ bất kỳ con nào), 0 (chờ cùng nhóm tiến trình)
   options: WNOHANG (chế độ không chặn), WUNTRACED (bắt cả khi con bị tạm dừng) */
```

Giá trị tham số PID	Ý nghĩa hoạt động
`pid > 0`	Chỉ chờ đúng tiến trình con có định danh PID bằng giá trị chỉ định.
`pid == 0`	Chờ bất kỳ tiến trình con nào cùng nhóm tiến trình (process group) với cha.
`pid == -1`	Chờ bất kỳ tiến trình con nào kết thúc (tương đương với lời gọi `wait()`).
`pid < -1`	Chờ bất kỳ con nào thuộc nhóm tiến trình có GID bằng giá trị tuyệt đối `|pid|`.

Cờ Tùy Chọn (Options)	Hành vi điều khiển
`WNOHANG`	Chế độ không chặn: Nếu chưa có con nào đổi trạng thái, hàm trả về ngay `0` thay vì treo cha.
`WUNTRACED`	Thu nhận trạng thái cả khi tiến trình con bị tạm dừng bởi tín hiệu (SIGSTOP, SIGTSTP).
`WCONTINUED`	Thu nhận trạng thái khi tiến trình con được đánh thức trở lại bởi tín hiệu SIGCONT (Linux 2.6.10+).

Giải Mã Cấu Trúc Trạng Thái Thoát (Wait Status Word)

Biến `status` trả về là một số nguyên, trong đó 2 byte thấp (16-bit) chứa thông tin kết thúc chi tiết:



Macro Kiểm Tra Trạng Thái	Ý nghĩa và cách trích xuất dữ liệu
`WIFEXITED(status)`	Trả về *true* nếu con kết thúc bình thường (qua `exit()`). Dùng `WEXITSTATUS(status)` lấy mã trả về (0-255).
`WIFSIGNALED(status)`	Trả về *true* nếu con bị ép dừng bởi tín hiệu. Dùng `WTERMSIG(status)` để lấy mã số tín hiệu; `WCOREDUMP(status)` kiểm tra core dump.
`WIFSTOPPED(status)`	Trả về *true* nếu con bị dừng lại. Dùng `WSTOPSIG(status)` để lấy mã số tín hiệu dừng.
`WIFCONTINUED(status)`	Trả về *true* nếu tiến trình con được khởi động lại thông qua nhận tín hiệu `SIGCONT`.

```
/* Ví dụ: Thu nhận và phân tích trạng thái kết thúc của tiến trình con */
int status;
pid_t pid = waitpid(child_pid, &status, 0);

if (WIFEXITED(status)) {
    // Con kết thúc chủ động bằng exit(code) hoặc lệnh return
    printf("Con kết thúc bình thường, mã thoát (exit code) = %d\n", WEXITSTATUS(status));
} else if (WIFSIGNALED(status)) {
    // Con bị buộc dừng do nhận tín hiệu bất thường (ví dụ SIGSEGV, SIGKILL)
```

NẠP CHƯƠNG TRÌNH MỚI VỚI EXECVE() & HỌ HÀM EXEC()

Thay Thế Không Gian Nhớ • Biến Thể exec(3) • Kết Thúc Tiến Trình exit()

PHẦN V: THỰC THI & HỦY BỎ

2.5 NẠP VÀ THỰC THI CHƯƠNG TRÌNH: LỜI GỌI EXECVE()

Lời gọi hệ thống `execve()` nạp một chương trình mới vào bộ nhớ của tiến trình hiện tại. Toàn bộ mã máy, dữ liệu, heap và stack cũ của tiến trình bị hủy hoàn toàn; chương trình mới bắt đầu chạy từ hàm `main()`. Định danh tiến trình (PID) vẫn được giữ nguyên.

```
#include <unistd.h>
int execve(const char *pathname, char *const argv[], char *const envp[]);
/* • Khi thành công: KHÔNG BAO GIỜ trả về (toàn bộ không gian nhớ bị mã mới thay thế)
   • Khi thất bại : Trả về -1, đặt mã lỗi errno (EACCES, ENOENT, ENOEXEC, E2BIG...) */
```

2.5.2 Phân Biệt Các Biến Thể Trong Họ Hàm Thư Viện exec()

Nhân Unix cung cấp duy nhất lời gọi hệ thống `execve()`, còn thư viện C chuẩn bổ sung họ hàm `exec()` nhằm mang lại sự tiện lợi khi lập trình dựa trên các tiền tố/hậu tố:

- **Chữ cái `p` (PATH):** Tự động tìm kiếm chương trình theo danh sách thư mục trong biến môi trường `$PATH` (không cần truyền đường dẫn tuyệt đối).
- **Chữ cái `l` (List):** Truyền danh sách đối số dạng biến đổi trực tiếp, kết thúc bằng con trỏ `NULL`.
- **Chữ cái `v` (Vector):** Truyền mảng các con trỏ xâu `argv[]`, kết thúc bằng phần tử `NULL`.
- **Chữ cái `e` (Environment):** Cho phép tùy biến mảng biến môi trường riêng `envp[]`.

Hàm	Chỉ định tệp	Định dạng tham số	Biến môi trường	Ví dụ gọi thực tế
`execve()`	Đường dẫn (pathname)	Mảng mảng `argv[]`	Tùy biến `envp[]`	`execve("/bin/ls", args, env);`
`execvp()`	Tên tệp (tìm $PATH)	Mảng mảng `argv[]`	Kế thừa từ cha	`execvp("ls", args);`
`execlp()`	Tên tệp (tìm $PATH)	Liệt kê `arg0, ..., NULL`	Kế thừa từ cha	`execlp("ls", "ls", "-l", NULL);`
`execv()`	Đường dẫn (pathname)	Mảng mảng `argv[]`	Kế thừa từ cha	`execv("/bin/ls", args);`
`execl()`	Đường dẫn (pathname)	Liệt kê `arg0, ..., NULL`	Kế thừa từ cha	`execl("/bin/ls", "ls", "-l", NULL);`
`execle()`	Đường dẫn (pathname)	Liệt kê `arg0, ..., NULL`	Tùy biến `envp[]`	`execle("/bin/ls", "ls", NULL, env);`

2.6 KẾT THÚC TIẾN TRÌNH: PHÂN BIỆT _EXIT() VÀ EXIT()

Tiến trình kết thúc qua hai hình thức: bất thường (do nhận tín hiệu hủy) hoặc bình thường (chủ động kết thúc):

```
#include <unistd.h>
void _exit(int status);  // Lời gọi hệ thống: Thoát tức thì (chỉ trả 8-bit thấp của status về cha)

#include <stdlib.h>
void exit(int status);   // Hàm thư viện chuẩn: Xả bộ đệm luồng I/O, chạy atexit() rồi gọi _exit()
```

Các bước dọn dẹp do hàm thư viện `exit()` thực hiện:

1. Kích hoạt lần lượt các hàm dọn dẹp (exit handlers) đã đăng ký qua `atexit()` theo thứ tự ngược.
2. Xả toàn bộ dữ liệu còn đọng trong các vùng đệm luồng chuẩn (flush stdio streams: `stdout`, `stderr`).
3. Gọi lời gọi hệ thống `_exit(status)` để kernel giải phóng bộ nhớ, đóng file descriptor.

3. GIAO TIẾP LIÊN TIẾN TRÌNH (INTERPROCESS COMMUNICATION - IPC)

Cơ Chế Tín Hiệu (Signals)

Tín hiệu là thông báo ngắt bất đồng bộ (asynchronous notification) do nhân Linux gửi tới tiến trình khi một sự kiện xảy ra. Tiến trình có thể bỏ qua, thi hành hành vi mặc định, hoặc bắt tín hiệu bằng hàm tự định nghĩa (Signal Handler).

ID	Tên Tín Hiệu	Hành vi mặc định	Sự kiện kích hoạt
`2`	`SIGINT`	Dừng tiến trình	Người dùng nhấn tổ hợp phím `Ctrl + C` từ bàn phím.
`9`	`SIGKILL`	Hủy diệt tức thì	Bắt buộc dừng tiến trình ngay lập tức (không thể chặn hoặc bắt giữ).
`11`	`SIGSEGV`	Dừng & Dump core	Lỗi truy cập vùng nhớ không hợp lệ (Segmentation Fault).
`14`	`SIGALRM`	Dừng tiến trình	Bộ đếm thời gian (timer) được kích hoạt bởi `alarm()` phát hỏa.
`17`	`SIGCHLD`	Bỏ qua (Ignore)	Tiến trình con bị tạm dừng hoặc đã kết thúc thực thi.

```c
/* Ví dụ: Đăng ký hàm bắt tín hiệu (Signal Handler) xử lý tổ hợp phím Ctrl + C */
#include <signal.h>
#include <stdio.h>

void sig_handler(int sig) {
    printf("\n[BẮT TÍN HIỆU] Đã nhận SIGINT (Ctrl+C). Bỏ qua hành vi thoát mặc định!\n");
}

// Trong hàm main():
signal(SIGINT, sig_handler);  // Đăng ký hàm xử lý tín hiệu SIGINT
// (Để gửi tín hiệu từ tiến trình khác: kill(pid, SIGINT);)
```

Cơ Chế Đường Ống Vô Danh (Anonymous Pipes)

Đường ống là kênh giao tiếp một chiều (unidirectional buffer) giữa hai tiến trình có quan hệ cha-con. Tiến trình ghi (Producer) đưa dữ liệu vào `fd[1]`; tiến trình đọc (Consumer) nhận dữ liệu từ `fd[0]`. Nếu ống rỗng, bên đọc sẽ bị chặn; nếu ống đầy (thường là 64KB), bên ghi sẽ chờ.

```c
/* Giao tiếp một chiều giữa Cha (Ghi) và Con (Đọc) qua Pipe vô danh */
#include <unistd.h>
int fd[2];
pipe(fd);  // fd[0]: Đầu ĐỌC (Read end) | fd[1]: Đầu GHI (Write end)

if (fork() == 0) {
    close(fd[1]);                    // Con chỉ đọc -> Đóng ngay đầu ghi fd[1]
    read(fd[0], buf, sizeof(buf));    // Đọc dữ liệu từ đường ống
    close(fd[0]);                    // Đóng đầu đọc sau khi dùng xong
} else {
    close(fd[0]);                    // Cha chỉ ghi -> Đóng ngay đầu đọc fd[0]
    write(fd[1], "Xin chào con", 13); // Đẩy thông điệp vào ống
    close(fd[1]);                    // Đóng đầu ghi sau khi hoàn tất
}
```

Chuyển Hướng Luồng Chuẩn Với dup() Và dup2()

`dup2(oldfd, newfd)` đóng `newfd` (nếu đang mở) và sao chép `oldfd` sang `newfd`. Thường áp dụng để nối đầu pipe vào đầu vào chuẩn `STDIN_FILENO (0)` hoặc đầu ra chuẩn `STDOUT_FILENO (1)` khi mô phỏng lệnh đường ống Unix (như `ls | wc -l`):

```c
/* Chuyển hướng đầu ra tiêu chuẩn (stdout) sang tệp tin */
```